COMPUTER VISION · INTERNSHIP PROJECT

Making YOLOv7 Explainable

Understanding *why* our detector fires — a heatmap-driven investigation into false positives

Duration: 4 weeks

Model: YOLOv7 (WongKinYiu official repository), fine-tuned on our data

Primary theme: Explainability & heatmap visualization (CAM family and beyond)

> This document tells you **what we want to achieve** and **why it matters**, and hands you tips where they save real time. It deliberately does **not** give you a copy-paste recipe. A large part of the value of this internship — for you and for us — is that you investigate, make choices, hit walls, and figure out how to get past them. Treat every "explore this" prompt as an invitation, not a test with one correct answer.

1 Why this project exists

We run fine-tuned YOLOv7 models in production to detect our objects. They work well overall — but they sometimes produce **false positives**: confident detections where there is no real object. A confidence number alone doesn't tell us *why* the model was fooled, so we can't fix the cause with any precision.

Your mission is to give us **eyes into the model**: a way to look at a false positive and see which part of the image the model was actually reacting to. Once we can see the "focus area" behind a wrong detection, we can reason about the cause — a misleading texture, a background pattern, a shadow, an artifact — and decide on a targeted fix later. That later fix is *not* your job for these four weeks. **Your job is the seeing.**

> **THE ONE-SENTENCE GOAL**
>
> Produce a reliable, repeatable way to generate heatmaps over YOLOv7 detections on our data, and use it to explain a set of real false positives — with an honest account of what the heatmaps do and do not tell us.

2 What success looks like

By the end, we'd love to see:

- A working heatmap pipeline for YOLOv7 that **you understand end to end** — not a black box you got running once.
- Heatmaps applied to a curated set of our real false positives, each with a short hypothesis about the cause.
- A clear-eyed writeup of the **limits** of the method: when a heatmap genuinely explains a detection, and when it only looks convincing.
- Documentation good enough that the next person can reproduce your results without asking you.

> **WHAT WE VALUE MOST**
>
> We are *more* impressed by "I tried X, it gave a misleading result for this reason, so I switched to Y" than by a single polished picture with no story behind it. Show us your reasoning, including the dead ends.

3 Background you'll want to build

You don't need to know all of this on day one — building this understanding *is* part of week one. But here's the map of the territory so you know what to go read about.

The core idea: Class Activation Mapping (CAM)

CAM-family methods turn a model's internal activations into a heatmap over the input image, highlighting the regions that most influenced a particular output. There is a whole family — Grad-CAM, Grad-CAM++, EigenCAM, HiResCAM, LayerCAM, XGradCAM and others — and they differ in how they weight the activations and whether they need gradients.

> **EXPLORE & DECIDE FOR YOURSELF**
>
> Some CAM methods are *gradient-free* (easy to run, but not class-discriminative) and some are *gradient-based* (they can answer "why *this class*?"). Find out which is which, and form your own opinion on which one is the right *starting* point versus the right tool for the actual investigation. There isn't a single right answer — we want your justification.

Why detection is harder than classification

Almost every CAM tutorial you'll find online is for a classifier that outputs one label per image. YOLOv7 is different: it outputs many boxes, each with a location, an "objectness" score, and per-class scores. That difference creates real questions you'll have to answer:

- When you compute a heatmap, **what output are you explaining** — the objectness, the class score, or the box coordinates?
- YOLOv7 detects at **three scales**. Which internal layers should the heatmap come from?
- A standard heatmap is computed for the **whole image**, not one box. What does that imply when you're trying to explain a single false positive?

> **HOLD THESE QUESTIONS IN YOUR HEAD**
>
> You are not expected to answer these on day one. But by the end of week two you should be able to explain each of them to us in your own words, with evidence from your own experiments.

4 The four-week arc

These are **themes and outcomes**, not a checklist. How you get to each week's outcome is up to you. If you find a better order, propose it — just tell us why.

WEEK 1 Orient & get one heatmap alive

research + first pixel on screen

Understand the CAM landscape and the specific challenges of applying it to a detector. But don't spend the whole week only reading — aim to get *one* heatmap rendered on a stock YOLOv7 with a public image before the week is out. A single working picture de-risks everything that follows and tells you the toolchain is sane.

Outcome: a short written summary of the methods (in your own words) + one heatmap image you generated yourself.

WEEK 2 YOLOv7 on COCO, breadth of methods

learn the knobs on neutral ground

Work on **pretrained YOLOv7 with public/COCO images first**, before touching our data. This lets you learn the method's behaviour without our dataset's quirks confusing the picture. Try more than one CAM variant and more than one "target". Build intuition for how the choices change the map.

Outcome: a comparison — same image, several methods and targets side by side — plus your explanation of the differences.

WEEK 3 Our data & the real investigation

from "it runs" to "it explains"

Now bring it to our fine-tuned model and our data. The goal here is not merely "make it run on our images" — it's to **collect a handful of genuine false positives and explain them**. Curate the FP examples first, then turn the heatmaps on them deliberately.

Outcome: a false-positive gallery — each FP with its heatmap and a one-line hypothesis about the cause.

WEEK 4 Document, reflect, present

make it reproducible & honest

Turn your work into something the team can use and trust. A README so the next person can reproduce it, a short presentation of findings, and — importantly — a candid section on the method's limitations and what you'd try next.

Outcome: a runnable, documented repo + a short slide deck of findings and limits.

PACE TIP

Prove the whole pipeline on **YOLOv7 first**. If you get curious about YOLOv9, treat it as a stretch goal for the very end — it has an architectural wrinkle (a dual detection head) that can eat days if you're not careful. Don't let it steal time from the actual investigation.

5 Tips & tricks (hard-won time savers)

These are genuine shortcuts around traps we know about. They're here to save you days, not to remove the thinking.

START WITH THE READY-MADE TOOLING

There is an open-source script written directly against the official YOLOv7 repo that produces heatmaps with no architecture surgery: z1069614715/objectdetection_script (look in the `yolo-gradcam` folder). Building on top of a working reference is smart, not cheating — but **read every line you run**. We will ask you to explain what it does.

PIN YOUR DEPENDENCY

The heatmap library (`grad-cam`) has changed its internals across versions. An older pinned version (`grad-cam==1.4.8`) is known to work with those scripts; newer ones can break them with cryptic attribute errors. If something fails mysteriously on the plumbing rather than your logic, suspect the version first.

THE LAYER INDICES IN ANY SCRIPT YOU FIND ARE PROBABLY NOT YOURS

Scripts ship with layer indices for *someone else's* model. YOLOv7's detection head records which internal layers feed it. **Explore how to discover the correct layers for your own trained weights** rather than copying numbers from a blog — hooking the wrong layers gives a real-looking but meaningless heatmap. (Hint: the detection module knows which layers feed it; a "second-to-last layer" fallback is a safe first attempt.)

> **KNOWN TRAP: GRADIENTS & IN-PLACE OPERATIONS**
>
> If you try to hook the detection head directly, you may hit an error about "a variable modified by an inplace operation." This is a well-known YOLOv7 gotcha. The ready-made scripts avoid it by hooking a feature layer *before* the head. If you go your own way and hit it, that's the cause — now you know where to look.

> **EVAL MODE, BUT KEEP GRADIENTS ON**
>
> Gradient-based CAM needs the model in `eval()` mode *and* gradients enabled. Wrapping inference in a "no gradients" block will silently give you nothing. If your gradient method produces a blank map, check this before anything else.

CHOOSING WHAT TO EXPLAIN

For our false-positive question, think carefully about whether you want the heatmap to explain the *class decision* ("why did it call this our object?") or the *box placement* ("why is the box here?"). They answer different questions. Decide which fits FP analysis and be ready to defend the choice.

> **EXPLORE BEYOND CAM**
>
> CAM is the easy on-ramp, but it isn't the only way to see a model's focus — and it has a real weakness for *per-detection* explanation. Investigate at least one non-CAM alternative (for example: occlusion sensitivity, or a detector-specific method like D-RISE). Even a small experiment comparing it against CAM on one stubborn false positive would be a standout contribution. We're leaving *which* one to you.

6 Thinking well about heatmaps

The single most valuable habit you can build in this project is **skepticism about your own pretty pictures**. A colourful heatmap is persuasive whether or not it's meaningful. Keep asking:

- Does the highlighted region actually correspond to *this* detection, or is it lit up for some other reason in the image?
- If I change how the heatmap is scaled/normalized, does my conclusion survive? (It often doesn't — find out for yourself.)
- Does the heatmap tell me *why* a region fooled the model, or only *where* it looked? (These are different, and the gap is where your interpretation comes in.)

> **THE QUESTION WE MOST WANT YOU TO ANSWER**
>
> For your false positives: **does the heatmap genuinely explain the wrong detection, or does it just show where there's visible structure?** A thoughtful, evidence-backed answer to this — even if the answer is "it depends, and here's when" — is the heart of a great result.

7 How to work with us

When you…	Please…
Get stuck for more than half a day	Come talk to us. Being stuck is normal; staying stuck silently isn't the goal.
Make a design choice (method, target, layer)	Write down *why*, even one line. Your reasoning is part of the deliverable.
Get a surprising or "too good" result	Be suspicious and try to break it before celebrating.
Want to change the plan	Propose it with a reason. Ownership is welcome.
Finish a week's outcome	Share it early, rough is fine. Frequent small check-ins beat one big reveal.

> **A NOTE ON USING AI ASSISTANTS AND THE INTERNET**
>
> Use every resource you like — that's real engineering. The only rule: **never run code you can't explain.** If you used something to get past a wall, understand it well enough to teach it back to us. That's how this becomes *your* learning and not just a finished task.

8 Starting points for your own research

Deliberately a short list — finding the rest is part of the work.

- **YOLOv7 (official):** github.com/WongKinYiu/yolov7
- **Ready-made YOLO heatmap scripts:** github.com/z1069614715/objectdetection_script (see `yolo-gradcam`)
- **The CAM library & its tutorials:** github.com/jacobgil/pytorch-grad-cam — it has an object-detection tutorial worth studying closely.
- **Search terms to get you going:** "Grad-CAM object detection", "EigenCAM YOLO", "Explaining YOLO Grad-CAM", "D-RISE detector saliency", "occlusion sensitivity object detection".

This guideline sets direction, not a script. The gaps in it are intentional — they're where your learning happens. When in doubt, make a reasoned choice, write down why, and bring it to us. Good luck, and enjoy the investigation.